

ABRA — systems audit, 2026-07-31

Every number below came from running something during this audit. Where a document and a measurement disagreed, the measurement won. Where I could not measure, I say so rather than infer.

Two findings in this report are **my own errors, caught during the audit** and recorded at the same severity as everything else. An audit that only finds other people's mistakes is not being run honestly.

Verdict

The data discipline is genuinely strong and the engine is genuinely well built. **The weakness is not in what this project knows — it is in what it enforces.** The tools to catch almost every failure here already exist, were written deliberately, produce correct answers, and are not wired to anything that fails.

The single most important number in this audit: `engine/provenance.js --strict` **exits 1 with 31 artifacts UNSAFE to quote, and nothing runs it.** The roadmap has recorded this as "the blocker on everything in section 3" since it was 28.

1. Data integrity

The stores are clean. Measured directly rather than trusted:

store	lines	unique ids	duplicates	missing id
<code>games.ladder.jsonl</code>	27,512	27,512	0	0
<code>games.bo3.jsonl</code>	5,175	5,175	0	0

The current guard is narrower than the failure it was written for, but it holds. `quality.js` opens a *named path* (`data/games.ladder.jsonl`), not a glob, so no experiment output can leak into the clean corpus. That is the structural property that matters, and it is correct.

What can still corrupt it, ranked:

- 1. Nothing found that silently corrupts the clean set.** Stated plainly as a negative finding — I looked for glob-based loading, for append-without-dedup, and for a second writer, and did not find one. That is a real result, not an absence of effort.
- 2. 23.7 GB of dead experiment output across 30 files** sits in `data/` (`games.h2h-*.jsonl` , up to 5.66 GB each). **Correctly gitignored**, and disk has 294 GB free, so the blast radius is **low** — this is untidiness, not a wrong-number risk. It does make `engine/provenance.js` glob 30 irrelevant files (`provenance.js:110` , `/^games\..*\jsonl$/`).

I initially over-reported item 2 as a data-integrity hazard. It is not; it is disk hygiene. The correction is recorded because mis-ranking a finding is itself a defect in an audit.

2. Single source of truth

The real divergence: speed multipliers

`board.js` derives **every** speed multiplier by calling the dex's own handlers — nothing typed:

```

item      dex.items.get(mon.item).onModifySpe
paralysis dex.conditions.get(status).onModifySpe      ("from the condition itself
                                                    rather than from a 0.5 written here")
abilities dex.abilities.get(ab).onModifySpe, weighted by population odds

```

medicham2-browser.js hardcodes the same three facts:

```

:720  if (m.status === 'par') s *= 0.5;
:718  if (m.item === 'choicescarf') s *= 1.5;    ... twA/twB > 0 -> s *= 2

```

Two implementations of one fact, which CLAUDE.md **explicitly forbids** — its own words: *"The MULTIPLIERS underneath them — Scarf $\times 1.5$, paralysis $\times 0.5$, Tailwind $\times 2$ — are the fact, and must be one function that both call."*

Removable? Not cheaply. medicham2-browser.js runs in the browser, where the Showdown dex is not available; that is a real constraint, not laziness. So the duplication stays and needs a check.

They agree today. test-engine-consistency.js asserts the Choice Scarf multiplier is exactly $\times 1.5$ in medicham2 and that board.js's switchFaster flips — but **it never compares the two numbers to each other**, and **paralysis and Tailwind are not checked at all**. A dex change or a Champions rule change would move board.js and leave medicham2 behind, silently.

(tag_dex.js:1209 already records that Champions' full-paralysis rate is 12.5% where the usual figure is 25% — so this format demonstrably does diverge from defaults.)

Other duplications, already known

where	state	removable
app/index.html embedded scorer	assigns 21 of 56 features; 35 silently zero	yes — render engine values, as scoreboard.html does
CHOMP champ-model.js	a separate damage engine in a separate repo	no — CHOMP is standalone by design; needs a cross-repo golden test
engine/graveyard/medicham-v2-singles.js	paralysis at 0.25 (pre-Gen-7)	already dead; correctly quarantined

3. Silent wrongness

Found: the only tautological assertion in the suite

tests/test-engine-consistency.js:89 was:

```

const pBare = P.positionFeatures(...)[pi];      // computed, never used
ok(true, 'position_features reads the same sheet path', '(shares medicham2 effSpeed)');

```

ok(true) **cannot fail**. It described a claim it did not test, in a file whose own header says *"NUMBER CHANGING proves everything."* I swept the whole suite for this pattern: **it is the only one**. (test-fragility.js:72 is a legitimate skip guard, not a fake assertion.) That is a good result for the suite and a bad one for this line.

FIXED, and the fix immediately earned itself: the replacement now asserts a declared Choice Scarf moves speedEdge, and reads **0.000** → **1.000**.

Found: a silent revert to a documented bug

position_features.js line 243 records a real past regression: it "compared raw st.sp while its own comment claimed to use effective speed. It did not."

The fixed code was:

```
try { sa = M.effSpeed(att, field, attSideTag); } catch (e) { sa = (att && att.st && att.st.sp) || 0; }
```

The catch reverts to exactly that bug — raw speed, no Scarf, no Tailwind, no paralysis — on any exception, with no counter. And it can fire. Measured:

input	result
normal	154
missing field	throws
null mon	throws
mon built without boosts	throws

All three are reachable from a caller that assembles a Pokémon by hand rather than via M.buildMon .

FIXED: STATS.speedFallbacks now counts it and is exported, matching this project's own idiom elsewhere (stats.jointFellBack , mew_farm's worker accounting). Verified o on the good path.

Found, in my own work during this audit

- 1. A malformed probe impersonated an engine failure. My first version of the replacement assertion used mkBoard (which leaves my side on the bench) where position_features needs mkActive . It read 0.000 → 0.000 and looked like a live bug. The helper's own comment says exactly this. Second time in one day a bad probe looked like a broken engine.
- 2. I mis-tested --strict as an environment variable when it is a --argv flag, concluded the gate was broken, and had to retract. The tool was correct.

Both are recorded because the class — a test that is wrong in the same direction as the hypothesis — is the most dangerous one in this repository and it does not stop applying to the auditor.

4. Provenance

For 4% of artifacts, yes. For 96%, no. Measured across all 70 JSON artifacts in data/ :

declares	count	share
a generation date	60	86%
a source script	25	36%
which filter produced it	3	4%

The project's central discipline is the clean/raw distinction — the Garbodor rule, that ~87% of the store is bots, forfeits and stubs. Three of seventy artifacts record which side of it they came from.

engine/provenance.js --strict : 31 UNSAFE, 6 possibly stale, 19 ok, 0 missing → exit 1.

The UNSAFE reason is uniform and specific: "OLDER THAN THE QUALITY FILTER — computed under different rules about what counts." Affected artifacts include value-net.json , xatu*.json , species-abilities.json , winrate-backtest.json .

The tool is right, complete, and honest about its own limit, which it prints: *"This checks what artifacts DECLARE about themselves. It cannot catch one that records a corpus it did not use — only re-running the generator can."*

5. Causal claims

I did not find an untested "X caused Y" propagated as fact. Stated as a negative finding.

The strongest candidate — MODELS.md's claim that greedy $\approx$ Nash *"because this meta is currently near-transitive at the coarse archetype level"* — is **well handled**: backed by a generated artifact (`meta-nash.json` , with exploitability), explicitly labelled *"an honest finding, not a win for mixing"*, and carrying a CI whose upper bound is stated to overturn it (*"under plausible resamples the meta is non-transitive"*).

One provenance gap inside it: `meta-nash.json` has **no** generated **field**. The claim is hedged correctly; the artifact underneath it cannot be dated.

The `because` clauses I sampled in MODELS.md are overwhelmingly *explanations of a design decision* ("kept separate because merging two kinds of evidence would corrupt a measured distribution"), not causal claims about the game. That is the right use of the word.

6. The handoff boundary

Convention, not structure. The evidence is direct:

```
engine/provenance.js:41    const STRICT = process.argv.includes('--strict');
engine/provenance.js:253  if (STRICT && (unsafe.length || OPTIN.length)) process.exit(1);
tests/run-all.js:54      GATES = [selftest, conformance, artifact_audit, validate_selfplay]
```

`provenance.js` **is not in** `GATES` . It can fail correctly and is never asked to. Its failure signal is opt-in behind a flag that no automated path passes.

By contrast, three checks *are* structurally enforced and they work: `selftest.js` , `conformance.js` , and `artifact_audit.js` — the last added 2026-07-30 precisely because "a check nobody acts on is not a check."

The boundary that is enforced: `magnemite.js` refuses to load a weight vector whose feature list disagrees with `board.js` , and `train_policy.js` refuses to start when a feature block receives no gradient. Both are structural, both fired during this session, both prevented a wrong number. That is the pattern the rest of the project should copy.

The rules

Each rule names the failure that produced it and the check that enforces it. **A rule with no check is marked as a preference.** That distinction is the point of the table.

#	Rule	Failure that produced it	Check that enforces it	Status
R1	A derived artifact must declare its source, its date, and its filter	3 of 70 artifacts declare a filter; 31 are UNSAFE to quote	<code>provenance.js --strict</code>	exists, NOT wired
R2	A fact may have two implementations only if a test compares their numbers	<code>board.js</code> derives speed multipliers; <code>medicham2</code> hardcodes 0.5/1.5/2	<code>test-engine-consistency.js</code> — covers Scarf, not paralysis or Tailwind	partial

#	Rule	Failure that produced it	Check that enforces it	Status
R3	No assertion may be a literal truth	<code>ok(true, ...)</code> in the consistency suite	grep for <code>ok(true, run in CI</code>	preference — no check yet
R4	A silent degradation must be counted	<code>effSpeed</code> catch reverted a documented bug with no counter	<code>STATS.speedFallbacks</code> , exported	fixed this audit
R5	A red test is fixed or waived by name, never filed	"known failure" normalised a rule violation for two days	CLAUDE.md, human discipline	preference — no check
R6	A long run must prove its lever is live before starting	two 1.5h runs where the switch block got zero gradient	<code>preflight.js</code> , gate in <code>train_policy.js</code>	enforced
R7	A model's weights must match the feature list they are scored with	53 → 56 features invalidated two vectors	<code>magnemite.js</code> throws on mismatch	enforced
R8	A probe that finds a bug must be checked against a known-good case first	two malformed probes impersonated engine failures in one day	none	preference — no check

Rules R3, R5 and R8 are preferences. They are stated as rules elsewhere in this project and are not enforced by anything. Calling them rules without saying so would be the same error this audit exists to find.

Grounding

- **R1** — provenance and lineage as a first-class artifact property is standard in data engineering; see *Data Provenance: Some Basic Issues* (Buneman, Khanna, Tan, FSTTCS 2000) for the formal treatment, and the "data lineage" requirement in Google's *Data Validation for Machine Learning* (Breck et al., SysML 2019), which specifically motivates schema-and-source checks that fail the pipeline rather than warn.
- **R2** — this is the *shared-nothing duplication* problem; the standard mitigation is a **contract test** between the two implementations (Fowler, *Integration Contract Test*). The golden-master pattern already used for the damage engine is the same idea.
- **R4** — counting rather than absorbing a degradation is the *fail-fast / fail-loud* principle (Shore, *Fail Fast*, IEEE Software 21(5), 2004).
- **R6** — this is a **pre-registration** check in the experimental sense: verify the manipulation actually manipulated something before spending the sample. The closest formal analogue is a *manipulation check* in experimental psychology (Perdue & Summers, *JMR* 1986).
- **R3, R5, R8** — I know of no literature specific to these. Stating that plainly rather than attaching a plausible-sounding citation.

Ranked by blast radius

Could put a wrong number in front of a reader:

1. **31 artifacts UNSAFE to quote, gate not wired** (R1). Anything published from `value-net.json`, `xatu*.json` or `winrate-backtest.json` is computed under superseded filter rules.
2. **Speed multipliers duplicated with no cross-check** (R2). A dex or format change moves one engine and not the other; every "who moves first" answer downstream would be quietly wrong.
3. `app/index.html` **scores 21 of 56 features**. A reader of the site sees numbers the bot did not compute. Already a failing test; the failure is honest and visible.

Merely untidy:

1. 23.7 GB of dead experiment output (gitignored, disk has 294 GB free).

2. `meta-nash.json` has no generation date.
3. `provenance.js` globs 30 irrelevant stores.

Fixed during this audit:

1. The tautological assertion (now discriminates, 0.000 → 1.000).
2. The uncounted silent fallback (now `STATS.speedFallbacks`).

What I could not verify, and left undone

- **Whether the 31 UNSAFE artifacts are actually wrong**, as opposed to merely undeclared. `provenance.js` says so itself: it checks what artifacts *declare*, and only re-running each generator can settle it. I did not re-run 31 generators. **The finding is "unprovable from disk", not "wrong".**
- **Whether medicham2's hardcoded multipliers currently disagree with the dex.** I established that nothing compares them and that `board.js` derives its own; I did not build the comparison, because `board.js` is off-limits while a training run is in flight. **R2 remains partial by my hand, not by omission.**
- **Adding** `provenance.js --strict to GATES` . This would turn the suite red immediately (31 artifacts). Under R5 that is either fixed or waived by name — and it is not mine to waive. **Recommended, not applied.**

Audit performed 2026-07-31 against commit `58ae510` , with a self-play training run in flight.

`engine/board.js` , `engine/magnemite.js` and `data/engine-data.js` were not modified, per that constraint. Suite after fixes: 38 passed, 1 failed (the `app/index.html` scorer, a pre-existing and honest failure).